

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2025 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Definitivat Model 2025	2
SUBIECTUL I (60 de puncte)	2
1. Tablouri bidimensionale (Matrice)	2
a) Exemple de declarare	2
b) Accesul la elemente și relațiile pe diagonale	2
c) Problemă completă (Suma elementelor de sub diagonala principală)	2
2. Dispozitive de stocare amovibile/detașabile	3
3. Programare: Tonomat k-perechi	3
4. Baze de date: Emisiuni Radio	5
SUBIECTUL al II-lea (30 de puncte)	6
1. Proiectarea unei activități didactice: Asaltul de idei (Brainstorming)	6
2. Test practic (Greedy)	6
II. Rezolvare Definitivat Varianta 2 (8 iulie 2025)	7
SUBIECTUL I (60 de puncte)	7
1. Structura de date: Listă simplu înlănțuită	7
a) Noțiuni preliminare: Parcurgerea nodurilor	7
b) Descrierea și exemplificarea etapelor pentru cele 4 operații de inserare	7
c) Exemplu de utilizare într-o problemă completă (Inserare Ordonată)	8
2. Sisteme de operare, rețele și securitate	10
a) Noțiuni preliminare:	10
b) Trei tipuri de amenințări informatice:	10
c) Două acțiuni ale unui antivirus la detectarea unui virus:	10
d) Exemplu de program antivirus cunoscut:	11
3. Programare: Generare etichete sală spectacol	11
4. Baze de date: Asociație Festivaluri	12
a) Modelul conceptual (Entități, Atribute, Relații, Forme Normale, Restricții)	12
b) Modelul Fizic (Structura tabelelor)	13
c) Adăugarea datelor de bază (Comenzi SQL)	13
d) Interogări SQL pentru extragerea informațiilor de sinteză	13
SUBIECTUL al II-lea (30 de puncte)	14
1. Proiectarea unei activități didactice (Alegere Secvență)	14
OPTIUNEA A: Secvența A (Metoda Divide et Impera)	14
OPTIUNEA B: Secvența B (Dispozitive de stocare a datelor)	14
2. Evaluarea rezultatelor școlare: Proba Scrisă	14
Itemi de evaluare:	14

I. Rezolvare Definitivat Model 2025

SUBIECTUL I (60 de puncte)

1. Tablouri bidimensionale (Matrice)

a) Exemple de declarare

1. Declarare cu inițializare cu date (alocare statică):

• C++:

```
int A[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

• Pascal:

```
const
    A: array[1..3, 1..3] of integer = (
        (1, 2, 3),
        (4, 5, 6),
        (7, 8, 9)
    );
```

2. Declarare cu alocare dinamică de memorie:

• C++:

```
int** A = new int*[linii];
for(int i = 0; i < linii; ++i) {
    A[i] = new int[coloane];
}
```

• Pascal:

```
type
    TLinie = array[1..100] of integer;
    PMatrice = ^array[1..100] of ^TLinie;
```

b) Accesul la elemente și relațiile pe diagonale

Fie o matrice pătratică A cu indicii de linie i și coloană j (1-indexed de la 1 la n):

• Accesul: A[i][j] (C++) sau A[i, j] (Pascal).

• Diagonala Principală: $i = j$.

- Deasupra (dreapta): $i < j$.
- Dedesubt (stânga): $i > j$.

• Diagonala Secundară: $i + j = n + 1$.

- Deasupra (stânga): $i + j < n + 1$.
- Dedesubt (dreapta): $i + j > n + 1$.

c) Problemă completă (Suma elementelor de sub diagonala principală)

• Enunț: Să se calculeze suma elementelor situate strict dedesubtul diagonalei principale a unei matrice pătratice de dimensiune n.

• Cod C++:

```
#include <iostream>
using namespace std;
int main() {
```

```
int n, A[100][100], sum = 0;
cin >> n;
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j) {
        cin >> A[i][j];
        if (i > j) sum += A[i][j];
    }
cout << "Suma: " << sum << endl;
return 0;
}
```

• **Cod Pascal:**

```
program SumaMatrice;
var
    n, i, j, sum: integer;
    A: array[1..100, 1..100] of integer;
begin
    read(n);
    sum := 0;
    for i := 1 to n do
        for j := 1 to n do
            begin
                read(A[i, j]);
                if i > j then sum := sum + A[i, j];
            end;
        end;
    writeln('Suma: ', sum);
end.
```

2. Dispozitive de stocare amovibile/detaşabile

- **Memorie internă vs externă:** Memoria internă (RAM, Cache) este rapidă, volatilă şi direct accesibilă procesorului. Memoria externă (HDD, SSD, Stick USB) este nevolatilă, are capacitate mare şi păstrează datele pe termen lung, comunicând cu procesorul prin intermediul controlerelor şi magistrelor de date.
- **Parametri:**
 1. **Viteza de transfer (Read/Write Speed):** Exprimată în MB/s sau GB/s. Determină rapiditatea accesului la date.
 2. **Capacitatea:** Volumul maxim de date stocat (GB/TB).
- **Dispozitive amovibile:**
 1. **Stick USB (Flash Drive):** Stocare pe cipuri de memorie Flash (EEPROM). Conectare prin port USB. Avantaj: Portabilitate extremă.
 2. **SSD Extern:** Stocare pe cipuri NAND Flash. Conectare prin USB-C sau Thunderbolt. Avantaj: Viteze mari de transfer.

3. Programare: Tonomat k-perechi

- **Descriere:** Subprogramul codGen calculează numărul de divizori ai lui n în $O(\sqrt{n})$. Programul principal citeşte intervalul, sortează capetele crescător şi parcurge elementele consecutiv, contorizând perechile care au numărul de divizori egal cu k .

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <algorithm>
using namespace std;
```

```

int codGen(int n) {
    int cnt = 0;
    for (int d = 1; d * d <= n; ++d) {
        if (n % d == 0) {
            cnt++;
            if (d * d < n) cnt++;
        }
    }
    return cnt;
}

int main() {
    ifstream fin("def2025.in");
    int k, x, y;
    if (!(fin >> k >> x >> y)) return 0;
    fin.close();

    int st = min(x, y);
    int dr = max(x, y);

    int count_pairs = 0;
    if (st < dr) {
        int prev_divs = codGen(st);
        for (int i = st; i < dr; ++i) {
            int curr_divs = codGen(i + 1);
            if (prev_divs == k && curr_divs == k) {
                count_pairs++;
            }
            prev_divs = curr_divs;
        }
    }

    cout << count_pairs << endl;
    return 0;
}

```

Soluție Pascal:

```

program TonomatKPerechi;
uses math;

var
    fin: text;
    k, x, y, st, dr, i, prev_divs, curr_divs, count_pairs: integer;

function codGen(n: integer): integer;
var
    d, cnt: integer;
begin
    cnt := 0;
    d := 1;
    while d * d <= n do
    begin
        if n mod d = 0 then
        begin
            cnt := cnt + 1;

```

```

        if d * d < n then
            cnt := cnt + 1;
        end;
        d := d + 1;
    end;
    codGen := cnt;
end;

begin
    assign(fin, 'def2025.in');
    reset(fin);
    read(fin, k, x, y);
    close(fin);

    st := min(x, y);
    dr := max(x, y);
    count_pairs := 0;

    if st < dr then
        begin
            prev_divs := codGen(st);
            for i := st to dr - 1 do
                begin
                    curr_divs := codGen(i + 1);
                    if (prev_divs = k) and (curr_divs = k) then
                        count_pairs := count_pairs + 1;
                    prev_divs := curr_divs;
                end;
            end;
        end;

        writeln(count_pairs);
    end.

```

4. Baze de date: Emisiuni Radio

Model conceptual:

- PIESA: id_piesa, titlu, compozitor, interpret, gen_muzical, an_aparitie, durata_secunde.
- EMISIUNE: id_emisiune, denumire, frecventa, data_start_grila, data_sfarsit_grila, ora_reper, minut_reper, descriere.
- DIFUZARE: id_difuzare, id_piesa, id_emisiune, data_difuzare, ora_difuzare, minut_difuzare.

Relații și reguli:

- PIESA 1:M DIFUZARE; o piesă poate fi difuzată de mai multe ori.
- EMISIUNE 1:M DIFUZARE; o emisiune poate include mai multe difuzări.
- Relația M:N dintre piese și emisiuni este normalizată prin DIFUZARE.
- 1NF**: câmpuri atomice, fără liste de difuzări într-un singur atribut.
- 2NF**: attributele depind complet de cheia primară a tabelului.
- 3NF**: datele piesei și datele emisiunii nu sunt duplicate în tabela difuzărilor.
- durata_secunde > 0, ora_difuzare în 0..23, minut_difuzare în 0..59.

Model fizic:

```

CREATE TABLE PIESA (
    id_piesa INT PRIMARY KEY AUTO_INCREMENT,
    titlu VARCHAR(150) NOT NULL,
    compozitor VARCHAR(120) NOT NULL,

```

```

interpret VARCHAR(150) NOT NULL,
gen_muzical VARCHAR(60),
an_aparitie INT,
durata_secunde INT CHECK (durata_secunde > 0)
);

CREATE TABLE EMISIUNE (
    id_emisiune INT PRIMARY KEY AUTO_INCREMENT,
    denumire VARCHAR(120) NOT NULL,
    frecventa VARCHAR(30) NOT NULL,
    data_start_grila DATE,
    data_sfarsit_grila DATE,
    ora_reper INT CHECK (ora_reper BETWEEN 0 AND 23),
    minut_reper INT CHECK (minut_reper BETWEEN 0 AND 59),
    descriere TEXT
);

CREATE TABLE DIFUZARE (
    id_difuzare INT PRIMARY KEY AUTO_INCREMENT,
    id_piesa INT NOT NULL,
    id_emisiune INT NOT NULL,
    data_difuzare DATE NOT NULL,
    ora_difuzare INT CHECK (ora_difuzare BETWEEN 0 AND 23),
    minut_difuzare INT CHECK (minut_difuzare BETWEEN 0 AND 59),
    FOREIGN KEY (id_piesa) REFERENCES PIESA(id_piesa),
    FOREIGN KEY (id_emisiune) REFERENCES EMISIUNE(id_emisiune)
);

```

SQL Afișare cronologică:

```

SELECT d.data_difuzare, d.ora_difuzare, d.minut_difuzare
FROM DIFUZARE d
JOIN PIESA p ON d.id_piesa = p.id_piesa
WHERE p.titlu = 'Anotimpurile - Vara'
    AND p.compozitor = 'Antonio Vivaldi'
    AND YEAR(d.data_difuzare) = YEAR(CURDATE())
ORDER BY d.data_difuzare ASC, d.ora_difuzare ASC, d.minut_difuzare ASC;

```

SUBIECTUL al II-lea (30 de puncte)

1. Proiectarea unei activități didactice: Asaltul de idei (Brainstorming)

- **Secvența A (Greedy):** Profesorul scrie pe tablă o problemă de optimizare (ex: Nunta lui Zamfira sau Problema Rucsacului - varianta continuă). Elevii sunt încurajați să propună orice idei de selectare a elementelor (după profit, după greutate, după raport). Ideile sunt colectate fără critică inițială, urmând a fi analizate și structurate ulterior pentru a defini tehnica Greedy.

2. Test practic (Greedy)

1. **Item 1:** Implementarea funcției de selecție a activităților în C++/Pascal. (30p)
2. **Item 2:** Argumentarea orală a corectitudinii algoritmului propus. (30p)
3. **Item 3:** Modificarea structurii de date pentru a obține o complexitate $O(N \log N)$. (30p)

II. Rezolvare Definitivat Varianta 2 (8 iulie 2025)

SUBIECTUL I (60 de puncte)

1. Structura de date: Listă simplu înlănțuită

a) Noțiuni preliminare: Parcurgerea nodurilor

O listă simplu înlănțuită este o structură de date dinamică formată din noduri alocate necontiguu în memoria Heap. Fiecare nod este format din:

- **Informația utilă** (datele stocate).
- **Legătura/Pointerul** către nodul următor (urm / next).

Structura de definire a unui nod:

C++:

```
struct Nod {
    int info;
    Nod* urm;
};
```

Pascal:

```
type
    PNod = ^TNod;
    TNod = record
        info: integer;
        urm: PNod;
    end;
```

Parcurgerea nodurilor: Pornind de la pointerul de start al listei (prim), se vizitează nodul curent și se înaintează la următorul prin reatribuirea pointerului de parcurgere cu legătura sa: $p = p \rightarrow \text{urm}$ (C++) sau $p := p^{\text{urm}}$ (Pascal), până când pointerul devine NULL / nil.

b) Descrierea și exemplificarea etapelor pentru cele 4 operații de inserare

Presupunem o listă cu 5 noduri: prim $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ [40] $\rightarrow$ [50] $\rightarrow$ NULL. Noul nod de inserat este indicat de pointerul nou, având valoarea 99.

1. Inserarea înainte de primul nod (la începutul listei)

- **Descriere:** Legătura noului nod este setată să indice către primul nod actual al listei. Pointerul de start al listei (prim) este apoi actualizat pentru a indica spre noul nod.

- **Cod C++:**

```
nou->urm = prim;
prim = nou;
```

- **Cod Pascal:**

```
nou^urm := prim;
prim := nou;
```

2. Inserarea după ultimul nod (la sfârșitul listei)

- **Descriere:** Se parcurge lista până când se ajunge la ultimul nod (cel care are $\text{urm} == \text{NULL}$). Legătura acestui ultim nod se setează să indice către nou, iar legătura lui nou devine NULL.

- **Cod C++:**

```
Nod* p = prim;
while (p->urm != nullptr) p = p->urm;
p->urm = nou;
nou->urm = nullptr;
```

- **Cod Pascal:**

```
p := prim;
while p^.urm <> nil do p := p^.urm;
p^.urm := nou;
nou^.urm := nil;
```

3. Inserarea înainte de un nod intermediar dat (ex. înaintea nodului cu valoarea 30)

- **Descriere:** Este necesară parcurgerea listei pentru a identifica nodul anterior celui căutat (nodul cu valoarea 20). Legătura lui nou este setată să indice către nodul căutat (nou->urm = anterior->urm). Legătura nodului anterior se actualizează să indice către nou.

- **Cod C++:**

```
Nod* ant = prim;
while (ant->urm != nullptr && ant->urm->info != 30) ant = ant->urm;
nou->urm = ant->urm;
ant->urm = nou;
```

- **Cod Pascal:**

```
ant := prim;
while (ant^.urm <> nil) and (ant^.urm^.info <> 30) do ant := ant^.urm;
nou^.urm := ant^.urm;
ant^.urm := nou;
```

4. Inserarea după un nod intermediar dat (ex. după nodul cu valoarea 30)

- **Descriere:** Se identifică nodul dat p (cel cu valoarea 30). Legătura lui nou preia adresa succesorului lui p (nou->urm = p->urm). Legătura lui p este setată să indice către nou.

- **Cod C++:**

```
Nod* p = prim;
while (p != nullptr && p->info != 30) p = p->urm;
nou->urm = p->urm;
p->urm = nou;
```

- **Cod Pascal:**

```
p := prim;
while (p <> nil) and (p^.info <> 30) do p := p^.urm;
nou^.urm := p^.urm;
p^.urm := nou;
```

c) Exemplu de utilizare într-o problemă completă (Inserare Ordonată)

- **Enunț:** Se citesc numere întregi până la introducerea valorii 0. Să se creeze o listă simplu înlănțuită prin inserare ordonată crescător, apoi să se afișeze elementele listei.

- **Implementare C++:**

```
#include <iostream>
using namespace std;

struct Nod {
    int info;
    Nod* urm;
};
```



```

void inserareOrdonata(Nod* &prim, int val) {
    Nod* nou = new Nod{val, nullptr};
    if (prim == nullptr || prim->info >= val) {
        nou->urm = prim;
        prim = nou;
    } else {
        Nod* curent = prim;
        while (curent->urm != nullptr && curent->urm->info < val) {
            curent = curent->urm;
        }
        nou->urm = curent->urm;
        curent->urm = nou;
    }
}

void afisare(Nod* prim) {
    for (Nod* p = prim; p != nullptr; p = p->urm) {
        cout << p->info << " ";
    }
    cout << endl;
}

int main() {
    Nod* prim = nullptr;
    int x;
    while (cin >> x && x != 0) {
        inserareOrdonata(prim, x);
    }
    afisare(prim);
    return 0;
}

```

• Implementare Pascal:

```

program ListaOrdonata;
type
    PNod = ^TNod;
    TNod = record
        info: integer;
        urm: PNod;
    end;
var
    prim: PNod;
    x: integer;

procedure inserareOrdonata(var prim: PNod; val: integer);
var
    nou, curent: PNod;
begin
    new(nou);
    nou^.info := val;
    nou^.urm := nil;
    if (prim = nil) or (prim^.info >= val) then
    begin
        nou^.urm := prim;
        prim := nou;
    end
end

```

```

else
begin
    curent := prim;
    while (curent^.urm <> nil) and (curent^.urm^.info < val) do
        curent := curent^.urm;
    nou^.urm := curent^.urm;
    curent^.urm := nou;
end;
end;

procedure afisare(p: PNode);
begin
    while p <> nil do
    begin
        write(p^.info, ' ');
        p := p^.urm;
    end;
    writeln;
end;

begin
    prim := nil;
    read(x);
    while x <> 0 do
    begin
        inserareOrdonata(prim, x);
        read(x);
    end;
    afisare(prim);
end.

```

2. Sisteme de operare, rețele și securitate

a) Noțiuni preliminare:

1. **Sistem de calcul:** Componenta fizică (hard) și cea logică (soft) ce conlucrează pentru a permite introducerea, stocarea, procesarea și livrarea datelor.
2. **Sistem de operare:** Programul de bază ce administrează memoria, procesele, componentele hardware și software, oferind o interfață utilizator-mașină.
3. **Rețea de calculatoare:** Un ansamblu de echipamente interconectate pentru partajarea de date, fișiere și resurse hardware.

b) Trei tipuri de amenințări informatice:

1. **Troian:** Se ascunde în programe utile; nu se multiplică singur; deschide breșe de acces (Backdoors).
2. **Ransomware:** Cripează fișierele și cere răscumpărare în criptomonede.
3. **Vierme (Worm):** Se propagă autonom prin rețele, exploatând breșe de securitate; consumă lățimea de bandă.

c) Două acțiuni ale unui antivirus la detectarea unui virus:

1. **Carantinarea:** Mutarea fișierului într-un folder criptat și izolat.
2. **Ștergerea / Curățarea:** Eliminarea semnăturilor virale sau ștergerea fișierului gazdă.

d) Exemplu de program antivirus cunoscut:

- Bitdefender Antivirus.

—

3. Programare: Generare etichete sală spectacol

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <algorithm>

using namespace std;

void eticheta(int s, int d, int &n) {
    int temp = d;
    int p = 1;
    while (temp > 0) {
        p *= 10;
        temp /= 10;
    }
    n = s * p + d;
}

int main() {
    int m, n;
    if (!(cin >> m >> n)) return 0;

    int A[101][101];
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            int val1, val2;
            eticheta(j, i, val1);
            eticheta(i, j, val2);
            A[i][j] = min(val1, val2);
        }
    }

    ofstream fout("def2025.txt");
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            fout << A[i][j] << (j == n ? "" : " ");
        }
        fout << "\n";
    }
    fout.close();

    return 0;
}
```

Soluție Pascal:

```
program GenerareEtichete;
uses math;

var
    m, n, i, j, val1, val2: integer;
```

```

A: array[1..100, 1..100] of integer;
fout: text;

procedure eticheta(s, d: integer; var n: integer);
var
    temp, p: integer;
begin
    temp := d;
    p := 1;
    while temp > 0 do
    begin
        p := p * 10;
        temp := temp div 10;
    end;
    n := s * p + d;
end;

begin
    readln(m, n);

    for i := 1 to m do
    begin
        for j := 1 to n do
        begin
            eticheta(j, i, val1);
            eticheta(i, j, val2);
            A[i, j] := min(val1, val2);
        end;
    end;

    assign(fout, 'def2025.txt');
    rewrite(fout);
    for i := 1 to m do
    begin
        for j := 1 to n do
        begin
            write(fout, A[i, j]);
            if j < n then
                write(fout, ' ');
            end;
            writeln(fout);
        end;
    end;
    close(fout);
end.

```

4. Baze de date: Asociație Festivaluri

a) Modelul conceptual (Entități, Atribute, Relații, Forme Normale, Restricții)

1. **FESTIVAL**: id_festival (PK), denumire (U, NOT NULL), tematica, site_oficial (NULL).
2. **TIP_EVENT**: id_tip (PK), denumire_tip (U, NOT NULL), spatiu_necesar, nr_max_participanti, varsta_minima.
3. **EVENIMENT**: id_eveniment (PK), id_festival (FK), id_tip (FK), denumire, descriere, data_oro_inceput, durata (CHECK > 0), locatie, categorie_public_tinta.

Relații:

- **FESTIVAL - EVENIMENT**: 1:M.
- **TIP_EVENTIMENT - EVENIMENT**: 1:M.

Forme Normale (3NF):

- **1NF**: Valori atomice în toate coloanele.
- **2NF**: Toate coloanele non-cheie depind integral de întreaga cheie primară (chei simple, nu compuse).
- **3NF**: Fără dependențe tranzitive (atributele non-cheie sunt direct dependente doar de cheia primară a tabelului lor).

b) Modelul Fizic (Structura tabelului)

Tabelă	Câmp	Tip de date	Restricții
FESTIVAL	id_festival	INT	PK, AUTO_INCREMENT
FESTIVAL	denumire	VARCHAR(100)	UNIQUE, NOT NULL
TIP_EVENTIMENT	id_tip	INT	PK, AUTO_INCREMENT
TIP_EVENTIMENT	denumire_tip	VARCHAR(50)	UNIQUE, NOT NULL
EVENIMENT	id_eventiment	INT	PK, AUTO_INCREMENT
EVENIMENT	id_festival	INT	FK -> FESTIVAL(id_festival)
EVENIMENT	id_tip	INT	FK -> TIP_EVENTIMENT(id_tip)
EVENIMENT	durata	INT	CHECK (durata > 0)

c) Adăugarea datelor de bază (Comenzi SQL)

```
INSERT INTO FESTIVAL (denumire, tematica, site_oficial)
VALUES ('EduCode 2025', 'Inovatie in predarea informaticii', NULL);
```

d) Interogări SQL pentru extragerea informațiilor de sinteză

Pentru a demonstra utilitatea modelului fizic propus, iată interogările SQL corespunzătoare cerințelor de sinteză din enunț:

1. Numărul de festivaluri în cadrul cărora s-au desfășurat cel puțin două evenimente de tip workshop:

```
SELECT COUNT(*) AS nr_festivaluri
FROM (
    SELECT e.id_festival
    FROM EVENIMENT e
    JOIN TIP_EVENTIMENT t ON e.id_tip = t.id_tip
    WHERE t.denumire_tip = 'workshop'
    GROUP BY e.id_festival
    HAVING COUNT(e.id_eventiment) >= 2
) AS subquery;
```

2. Tipurile de evenimente care nu s-au desfășurat în cadrul niciunui festival în ultimii doi ani:

```
-- Varianta bazată pe subinterogare cu NOT IN
SELECT id_tip, denumire_tip
FROM TIP_EVENTIMENT
WHERE id_tip NOT IN (
    SELECT DISTINCT id_tip
```

```
FROM EVENIMENT  
WHERE data_ora_inceput >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR)  
);
```

SUBIECTUL al II-lea (30 de puncte)

1. Proiectarea unei activități didactice (Alegere Secvență)

OPTIUNEA A: Secvența A (Metoda Divide et Impera)

- **Metoda: Problematizarea.**
- **Caracteristici:** Stimulează gândirea analitică prin crearea unei situații-problemă; încurajează participarea activă.
- **Scenariu:** Profesorul propune determinarea maximului dintr-un vector prin divizarea repetată în subvectori simetrici, ghidând elevii să descopere singuri principiul recursivității.

OPTIUNEA B: Secvența B (Dispozitive de stocare a datelor)

- **Metoda: Mozaicul (Jigsaw).**
- **Caracteristici:** Dezvoltă spiritul de echipă; fiecare elev este responsabil de asimilarea și predarea unei subteme.
- **Scenariu:** Elevii sunt împărțiți în grupe de experți (HDD, SSD, Cloud, Unități de măsură), studiază suportul tehnic, iar apoi predau subtemele în grupul de bază.

2. Evaluarea rezultatelor școlare: Proba Scrisă

- **Avantaje:** Evaluare obiectivă, economie de timp, reducerea factorului stres.
- **Limită:** Nu poate evalua deprinderile practice pe calculator.

Itemi de evaluare:

Secvența A (Alegere Multiplă): Alegerea metodei optime pentru căutarea rapidă a unei valori într-un tablou sortat. Răspuns: Căutarea binară.

Secvența B (Răspuns Scurt): Diferența dintre HDD și SSD. Răspuns: Viteza de transfer și rezistența la șocuri (fără piese mobile).